

LAB EXPERIMENT-3

Aim :To study and understand the compilation and debugging of C programs using the GNU Debugger (GDB) and to analyze the implementation of data structures such as Stack, Queue, and Linked List along with their memory behavior and addressing mechanisms.

Theory :

In this experiment, we use the GNU Debugger (GDB) to compile and debug C programs. Debugging helps in understanding how a program executes internally and how memory is utilized.

1.Stack :

- Stack follows LIFO (Last In First Out) principle.

Operations:

- Push (insert element)
 - Pop (remove element)
 - Peek (view top element)
- Stored in stack memory.
 - Accessed using a variable like top.

```
#include <stdio.h>
#define SIZE 5
int stack[SIZE];
int top = -1;
void push(int value) {
    if (top == SIZE - 1) {
        printf("Overflow\n");
        return;
    }
    stack[++top] = value;
}
void pop() {
    if (top == -1) {
        printf("Underflow\n");
        return;
    }
    top--;
}
int main() {
    push(10);
    push(20);
    pop();
    return 0;
}
```

2.Queue :

- Queue follows FIFO (First In First Out) principle.
- Operations:
 - Enqueue (insert)
 - Dequeue (remove)
- Implemented using arrays.
- Uses indexed addressing with front and rear.

```
#include <stdio.h>
#define SIZE 5

int queue[SIZE];
int front = -1, rear = -1;

void enqueue(int value) {
    if (rear == SIZE - 1) {
        printf("Overflow\n");
        return;
    }
    if (front == -1) front = 0;
    queue[++rear] = value;
}

void dequeue() {
    if (front == -1 || front > rear) {
        printf("Underflow\n");
        return;
    }
    front++;
}

int main() {
    enqueue(1);
    enqueue(2);
    dequeue();
    return 0;
}
```

3.Linked List :

- A dynamic data structure where elements are stored in nodes.

Each node contains:

- Data
 - Pointer to next node
- Uses heap memory allocation (malloc).

Operations:

- Insertion
- Deletion
- Traversal

```

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* next;
};

struct Node* head = NULL;

void insert(int value) {
    struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = head;
    head = newNode;
}

void display() {
    struct Node* temp = head;
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
}

int main() {
    insert(10);
    insert(20);
    display();
    return 0;
}

```

Procedure :

Step 1 : Write separate C programs for Stack, Queue, and Linked List with their respective operations.

Step 2 : Compile each program using a C compiler

Step 3 : Open terminal and run the following command to start debugging:

`gdb ./program_name`

Step 4 : Set breakpoint and execute step-by-step:

`break main`

`run`

`step / next`

Step 5 : Observe variables and memory behavior using:

`print variable_name`

`continue`

Observation :

For stack :

```
Breakpoint 2, push (value=10) at stackds.c:8
8      if (top == SIZE - 1) {
(gdb) n
12      top++;
(gdb) n

Breakpoint 1, push (value=10) at stackds.c:13
13      stack[top] = value; // <-- Put breakpoint here
(gdb) n
14      }
(gdb) n
main () at stackds.c:26
26      (20);
(gdb) n

Breakpoint 2, push (value=20) at stackds.c:8
8      if (top == SIZE - 1) {
(gdb) n
12      top++;
(gdb) n

Breakpoint 1, push (value=20) at stackds.c:13
13      stack[top] = value; // <-- Put breakpoint here
(gdb) n
14      }
(gdb) p top
$1 = 1
(gdb) p stack
$2 = {10, 20, 0, 0, 0}
(gdb) p stack[0]
$3 = 10
(gdb) p stack[1]
$4 = 20
(gdb) print &stack
$5 = (int (*)[5]) 0x555555580030 <stack>
(gdb) p &stack[1]
$6 = (int *) 0x555555580034 <stack+4>
(gdb) p &stack[0]
$7 = (int *) 0x555555580030 <stack>
(gdb) ATTENTION: Use &stack
```

For Queue :

```
1 //queue.c
nitdelhi@nitdelhi-HP-Elite-Tower-600-G9-Desktop-PC:~$ nano queue.c
nitdelhi@nitdelhi-HP-Elite-Tower-600-G9-Desktop-PC:~$
nitdelhi@nitdelhi-HP-Elite-Tower-600-G9-Desktop-PC:~$ gcc -std=c99 -g -o test queue.c
nitdelhi@nitdelhi-HP-Elite-Tower-600-G9-Desktop-PC:~$ gdb ./test
GNU gdb (Ubuntu 15.0.50.20240403-Dubautel) 15.0.50.20240403-g1t
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software; you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.
For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./test...
(gdb) "[[200-b enqueue
Undefined command: "[". Try "help".
(gdb) b enqueue
Breakpoint 1 at 0x1290: file queue.c, line 10.
(gdb) run
Starting program: /home/nitdelhi/test

This GDB supports auto-downloading debuginfo from the following URLs:
<https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, enqueue (value=10) at queue.c:10
10      if (rear == SIZE - 1) {
(gdb) n
15      if (front == -1)
(gdb) n
16          front = 0;
(gdb) n
18      rear++;
(gdb) n
19      queue[rear] = value; // <-- Breakpoint can be set here
(gdb) n
20      ("Enqueued: %d\n", value);
(gdb) n
Enqueued: 10
21      }
(gdb) n
main () at queue.c:52
52      (30); // 3rd value
(gdb) n
```

```
16      front = 0;
(gdb) n
18      rear++;
(gdb) n
19      queue[rear] = value; // <-- Breakpoint can be set here
(gdb) n
20      ("Enqueued: %d\n", value);
(gdb) n
Enqueued: 20
21      }
(gdb) n
main () at queue.c:52
52      (30); // 3rd value
(gdb) n

Breakpoint 1, enqueue (value=30) at queue.c:10
10      if (rear == SIZE - 1) {
(gdb) n
15      if (front == -1)
(gdb) n
18      rear++;
(gdb) n
19      queue[rear] = value; // <-- Breakpoint can be set here
(gdb) n
20      ("Enqueued: %d\n", value);
(gdb) n
Enqueued: 30
21      }
(gdb) n
main () at queue.c:52
52      (30); // 3rd value
(gdb) n
```

```

19     queue[rear] = value; // <-- Breakpoint can be set here
(gdb) n
20     ("Enqueued: %d\n", value);
(gdb) p queue
$1 = {10, 20, 30, 0, 0}
(gdb)
$2 = {10, 20, 30, 0, 0}
(gdb) p &queue[0]
$3 = (int *) 0x55555558030 <queue>
(gdb) p &queue[1]
$4 = (int *) 0x55555558034 <queue+4>
(gdb) p &queue[2]
$5 = (int *) 0x55555558038 <queue+8>
(gdb)

```

For Linked List :

```

nitdelhi@nitdelhi-HP-Elite-Tower-600-G9-Desktop-PC:~$ nano linkedlist.c
nitdelhi@nitdelhi-HP-Elite-Tower-600-G9-Desktop-PC:~$ gcc -std=c99 -g -o test linkedlist.c
nitdelhi@nitdelhi-HP-Elite-Tower-600-G9-Desktop-PC:~$ gdb ./test
GNU gdb (Ubuntu 15.0.50.20240403-0ubuntu1) 15.0.50.20240403-g
Copyright (C) 2024 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from ./test...
(gdb) break 5
Breakpoint 1 at 0x1a1: file linkedlist.c, line 5.
(gdb) run
Starting program: /home/nitdelhi/test

Breakpoint 1, main () at linkedlist.c:5
5      insert(10);
(gdb)

```

```

Breakpoint 1, main () at linkedlist.c:5
5      insert(10);
(gdb) step
insert (value=10) at linkedlist.c:12
12     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
(gdb) next
13     newNode->data = value;
(gdb) next
14     newNode->next = head;
(gdb) next
15     head = newNode;
(gdb) next
16     }
(gdb) next
main () at linkedlist.c:6
6      insert(20);
(gdb) print head
$1 = (struct Node *) 0x555555592a0
(gdb) print *head
$2 = {data = 10, next = 0x0}
(gdb) next
7      display();
(gdb) next
8      return 0;
(gdb) step
display () at linkedlist.c:19
19     struct Node* temp = head;
(gdb) print head
$3 = (struct Node *) 0x555555592a0
(gdb) print *head
$4 = {data = 10, next = 0x0}

```

```

(gdb) continue
Continuing.
10

Breakpoint 1, main () at linkedlist.c:6
6      insert(20);
(gdb) step
insert (value=20) at linkedlist.c:12
12     struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
(gdb) next
13     newNode->data = value;
(gdb) next
14     newNode->next = head;
(gdb) next
15     head = newNode;
(gdb) next
16     }
(gdb) next
main () at linkedlist.c:7
7      display();
(gdb) print head
$5 = (struct Node *) 0x555555592c0
(gdb) print *head
$6 = {data = 20, next = 0x555555592a0}
(gdb) print *head->next
$7 = {data = 10, next = 0x0}
(gdb) continue
Continuing.
20 10
[Inferior 1 (process 12345) exited normally]
(gdb)

```

During debugging using the GNU Debugger (GDB), the execution of Stack,

Queue, and Linked List programs was observed step-by-step. It was seen that the stack follows LIFO behavior with elements inserted and removed from the top, while the queue follows FIFO behavior using front and rear indexing. In contrast, the linked list uses dynamic memory allocation, where nodes are stored in heap memory and connected through pointers. Using GDB commands like print, the values and memory addresses of variables were examined, showing that arrays (stack and queue) use contiguous memory, whereas linked list nodes are stored in non-contiguous memory locations.

Conclusion :

The experiment was successfully performed using the GNU Debugger (GDB) to compile and debug programs for Stack, Queue, and Linked List. It helped in understanding the working principles of different data structures, including LIFO and FIFO operations, as well as dynamic memory allocation using pointers. The use of GDB enabled step-by-step execution and observation of variables and memory addresses, providing a clear understanding of memory layout and addressing mechanisms in computer architecture.